Software Engineering

Chapter 8
Design Concepts

Based on Pressman's Software Engineering: A Practitioner's Approach, 7/e

Complete Study Notes — Topic by Topic

■ What is Software Design?

The Big Picture

Software design is the process of **converting user requirements into a blueprint** that developers can actually implement. Think of it like an architect drawing plans before a building is constructed — you don't just start laying bricks without a plan.

It sits between requirements analysis and coding in the software development process:

Requirements → Design → Implementation

It's Iterative

Design is not a one-time thing. You start with a high-level rough design, then with each iteration you go deeper and add more detail. Early iterations answer *'what does the system look like broadly?'* and later ones answer *'how exactly does each piece work?'*

What Does the Design Model Show?

- **Software Architecture** — the overall structure; how major components are organized
- **Interfaces** — how components talk to each other and how users interact with the system
- **Components** — the individual building blocks and their internal details

The Four Types of Design

Think of these as zoom levels — from the widest view (architecture) down to the finest detail (component).

Design Type	What it does	Key Diagrams
Data/Class Design	Defines classes, attributes, objects and relationships	Class diagram, CRC, collaboration diagram
Architectural Design	Defines the overall framework — standards, distributed, etc.	Class diagram, CRC, DFD, control flow
Interface Design	Defines how data flows between humans and the system	Use case, state transition, sequence, DFDs
Component Level Design	Defines the procedural details of each component	DFD, sequence, state transition, class diagram

■ Good Software Design Characteristics

Part 1 — Quality Guidelines

- **Modular:** Logically partitioned into elements or subsystems. Don't build one giant blob.
- **Distinct Representations:** Separately represent data, architecture, interfaces, and components.
- **Appropriate Data Structures:** Use the right data structure for the right job.
- **Independent Functional Characteristics:** Components should do their job without heavy dependence on others.
- **Reduced Interface Complexity:** Connections between components should be as simple as possible. Complex interfaces = more bugs.
- **Derived Using a Repeatable Method:** Follow a method driven by requirements analysis — not gut feeling.
- **Understandable Notation:** Expressed in a way others can read — diagrams, models, pseudocode.

Part 2 — Design Principles

- **Avoid Tunnel Vision:** Don't get so focused on one part that you miss the bigger picture. Always consider alternatives.
- **Traceable to Analysis:** Every design decision should trace back to a requirement.
- **Don't Reinvent the Wheel:** If a solution already exists (patterns, libraries), use it.
- **Minimize Intellectual Distance:** The design should mirror the real-world problem as closely as possible.
- **Uniformity and Integration:** The design should look and feel consistent throughout.
- **Accommodate Change:** Design with the future in mind — requirements will change.
- **Degrade Gracefully:** When something goes wrong, the system should fail softly and handle errors gracefully.
- **Design ≠ Coding:** Design is about what and how at a high level — coding is the actual implementation.
- **Assess Quality During Creation:** Review and assess as you go — fixing problems early is much cheaper.
- **Review for Conceptual Errors:** Review for semantic errors — logical mistakes in how concepts relate.

Part 3 — Attributes for Assessing Design Quality

Attribute	What it means
Functionality	Does it do what it's supposed to? Is it secure?
Usability	Is it human-friendly? Consistent? Well-documented?
Reliability	How severe are failures? No single point of failure?

Performance	Speed, throughput, resource usage, response time
Supportability/Scalability	Can the system grow? Are limitations minimal?
Maintainability	Easy to install, update, and maintain?

■ How to Make an Effective Design

1. Problem Partitioning

Break a big complex problem into smaller, manageable pieces. Instead of trying to design the entire system at once, you define separate functions, modules, and sub-products that could operate independently.

> **Real life analogy:** Building a university portal is overwhelming as one problem. But break it into — Login System, Student Dashboard, Fee Portal, Result Module, Timetable Module — and suddenly each piece is manageable on its own.

2. Abstraction

Abstraction means **show only what's necessary at a given level, hide everything else.** Like Google Maps — zoomed out you see countries, zoom in you see roads, zoom further you see street names.

Higher level = more abstraction (hide implementation). Lower level = less abstraction (show details).

Two Types of Abstraction:

- **Procedural Abstraction:** A named sequence of instructions whose internal details are hidden. E.g., *login()* — you don't care how it checks the password internally.
- **Data Abstraction:** A named collection of data describing an object, hiding how it's stored. E.g., *student.getName()* without caring about internal storage.

3. Top-Down vs Bottom-Up

	Top-Down	Bottom-Up
Starting point	Whole system	Individual modules
Direction	Abstract → Detail	Detail → Abstract
Used when	Building fresh from requirements	Integrating existing parts
Example	Designing an app from scratch	Integrating payment gateway + auth library + existing DB

■ Fundamental Design Concepts

> This is the heart of Chapter 8. There are 11 fundamental concepts.

1. Abstraction

In modular design, every level of the system has its own abstraction. Higher levels → more abstract. Lower levels → show implementation.

- **Data Abstraction** — hides how data is stored (e.g. a Student object)
- **Procedural Abstraction** — hides how a function works internally (e.g. login())
- **Control Abstraction** — hides the control mechanism (e.g. sort() without knowing the algorithm)

2. Architecture

The **overall structure of the software** — how major components are organized and how they interact. Think of it as the skeleton of the system.

Style	Description	Example
Data-centered	Central data store accessed by multiple components	Database-driven apps
Data-flow	Data flows through a series of transformations	Compilers, pipelines
Call-and-return	Components call each other in a hierarchy	Traditional procedural programs
Object-oriented	System built around objects and their interactions	Most modern apps
Layered	System divided into layers, each serving the one above	OS, network protocols

3. Separation of Concerns

Any complex problem becomes easier if you split it into pieces that can be solved independently. A 'concern' is any feature or behavior that matters to a stakeholder.

> **Example:** In a banking app — user authentication, transaction processing, notification system, and fraud detection are all separate concerns. Changes in one should not break another.

This concept is the **parent idea** behind modularity, refinement, functional independence, and aspects.

4. Modularity

The direct implementation of separation of concerns. Software is broken into **separately named, addressable components called modules** which are then integrated to fulfill requirements.

Advantages: Easier to understand, easier to develop (parallel teams), cheaper to build, easier to maintain.

> **■■ The Trade-off: Too few modules = complex and hard to understand. Too many modules = integration cost explodes. There is an optimal number of modules where total cost is minimized.**

5. Information Hiding

Each module should **hide its internal details** behind a controlled interface. What gets hidden: the algorithm, internal data structures, external interface details, resource allocation policies.

> **Real example:** You use ArrayList in Java. You don't know how it manages memory. You just call .add(), .get(), .remove(). That's information hiding.

- Changes inside a module don't affect other modules
- Encourages communication through clean, controlled interfaces
- Discourages use of global data (hidden dependencies)
- Results in higher quality, more maintainable software

6. Functional Independence

Design each module so that it **does one specific job and has minimal interaction with other modules.** Independence is measured using two criteria:

Cohesion — How focused is a module on doing a single task?

Level	Type	Description
Worst ↓	Coincidental	Module does random unrelated things
	Logical	Groups similar but unrelated functions
	Temporal	Groups things that happen at the same time
	Procedural	Groups steps that follow a sequence
	Communicational	Works on same data
	Sequential	Output of one part feeds next
Best ✓	Functional	Does exactly one well-defined task

Coupling — How dependent are modules on each other?

Level	Type	Description
Worst ↓	Content	One module directly modifies another's internals
	Common	Modules share global data
	Control	One module controls flow of another
	Stamp	Passes complex data structures
Best ✓	Data	Only simple data passed through parameters

■ Golden Rule: High Cohesion + Low Coupling = Good Design

7. Refinement

Progressively elaborating the details of your design. You start with a high-level abstract statement and keep breaking it down until every detail is clear enough to implement. Abstraction and refinement are **complementary** — abstraction hides detail, refinement reveals it step by step.

Stepwise Refinement Example — Opening a Door:

Level 1 (Abstract)	Level 2 (Refined)	Level 3 (Further Refined)
open	• walk to door • reach for knob • open door • walk through • close door	• repeat until door opens: - turn knob clockwise - if knob doesn't turn: find key, insert, unlock - pull/push door • end repeat

8. Aspects

About **cross-cutting concerns** — requirements that affect multiple parts of the system simultaneously. Requirement A crosscuts Requirement B if you cannot satisfy B without considering A.

> **In simple words:** An aspect is a requirement that follows you everywhere in the system, like a shadow. No matter which feature you're designing, it's always there.

Example (SafeHomeAssured.com): Requirement A = camera access. Requirement B = all users must be validated. B crosscuts A — you can't design camera access without considering validation. So B* (design of validation) is an **aspect**.

Aspect	Why it's everywhere
Security/Authentication	Every feature needs to check if user is logged in
Logging	Every action needs to be recorded
Error Handling	Every module can fail
Internet connectivity check	Every feature in a mobile app needs network

9. Refactoring

Restructuring existing code/design to **improve its internal structure without changing its external behavior.** The system does the same thing before and after — but internally it's cleaner, simpler, and more maintainable.

- Refactoring is **NOT** adding new features
- Refactoring is **NOT** fixing bugs
- It's purely about improving the design/code structure

> **Real example:** A 200-line function doing 10 different things refactored into 10 separate focused functions of 20 lines each. Behavior is identical — but now it's readable, testable, and maintainable.

10. OO Design Concepts

Three types of design classes:

Class Type	Description	Example
Entity Classes	Represent system data / real world objects	Student, Course, Payment
Boundary Classes	Interface between system and actors (users/external)	LoginScreen, DashboardPage
Controller Classes	Mediate between boundary and entity classes	LoginController, PaymentProcessor

Core OO Concepts:

- **Inheritance** — subclasses automatically get all responsibilities of the superclass
- **Polymorphism** — same operation behaves differently based on the object; greatly reduces design complexity when extending
- **Messages** — objects communicate by sending messages that trigger behavior

11. Design Classes

As you move from analysis to design, classes get refined:

- **Analysis classes** → refined into entity classes during design
- **Boundary classes** → designed to represent how entity data is shown to users
- **Controller classes** → manage creation/update of entities, instantiation of boundary objects, complex communication, and data validation

Quick Summary — All 11 Fundamental Concepts

Concept	One Line Summary
Abstraction	Hide details, show only what's needed at each level
Architecture	Overall skeleton/structure of the system
Separation of Concerns	Split complex problems into independent pieces
Modularity	Implement separation via named, addressable modules
Information Hiding	Hide internals behind controlled interfaces
Functional Independence	One job per module, measure via cohesion & coupling

Refinement	Progressively elaborate design from abstract to detailed
Aspects	Handle cross-cutting concerns that affect multiple modules
Refactoring	Improve internal structure without changing behavior
OO Concepts	Entity, boundary, controller classes + inheritance, polymorphism
Design Classes	Refined analysis classes with full design detail

■ The Design Model & Its Elements

The design model is the **complete blueprint of the system** — it's the output of the entire design process. It has 5 elements.

1. Data Elements

About **how data is structured and stored** in the system. Works at two levels:

- **High level (abstract):** Define data from the user's point of view — what data exists, how it relates, what it means (like an ER diagram).
- **Low level (implementation):** Refined into actual storage — tables in a relational DB, data structures in code.

> **Why it matters:** Data architecture directly impacts software architecture. Good data storage can lead to data warehousing → data mining → business insights. Good data design has a ripple effect all the way to business success.

2. Architectural Elements

Shows the **overall view of the entire software system** — the big picture. Derived from three sources:

- **Application domain knowledge** — what kind of system is being built
- **Requirements elements** — DFDs, collaboration diagrams, relationship diagrams
- **Available architectural styles** — data-centered, OO, layered, etc.

Usually shown as a set of interconnected subsystems, each derived from a work package in requirements. Think of it like a **city map** — you see all districts and how they connect, but not the details inside each building.

3. Interface Elements

Defines **how information flows into and out of the system**, and how it's communicated between internal modules.

Type	What it is	Example
UI Design	How the user interacts with the system	Login screen, dashboard, forms
External Interface	How the system talks to outside systems/devices	Payment gateway, SMS API, hardware sensors
Internal Interface	How components talk to each other inside the system	Module A passing data to Module B

Interface design covers: **Aesthetic elements** (layout, color, visuals) and **Ergonomic elements** (navigation flow, information placement). External interfaces must include input validation and security. Tools like Figma and Adobe XD are used.

4. Component Elements

If architecture is the city map, component design is the **detailed blueprint of each individual building.** Describes the internal details of each software component:

- **Data structures** — what local data does this component hold?
- **Algorithmic details** — what process/logic runs inside? (shown via flowcharts, activity diagrams, pseudocode)
- **Interface** — what operations does this component expose to the outside?

> **Real example — PaymentProcessor component:** Data: transaction amount, card details, status. Algorithm: validate card → contact bank → confirm/reject → update DB. Interface: processPayment(), refund(), getStatus().

5. Deployment Elements

Shows the **physical environment** where the software will actually run — distributed or standalone.

Deployment diagrams show physical nodes (servers, devices, cloud), which components are deployed where, and how nodes communicate.

> **Example — Web app deployment:** Web server node → frontend. Application server node → business logic. Database server node → data. All connected via a network.

How All 5 Elements Relate — Hospital Analogy

Design Element	Hospital Analogy
Data Elements	Patient records system — how data is stored
Architectural Elements	Overall hospital layout — which departments exist and where
Interface Elements	Reception desk, signage — how patients interact with staff
Component Elements	Detailed design of each room/department internally
Deployment Elements	Which building, which floor, which city each department is in

---

End of Chapter 8 — Design Concepts

Based on Pressman's Software Engineering: A Practitioner's Approach, 7/e